

RAKSHASAFE

AI-Powered Women Safety and Disaster Emergency Response System

CHAPTER 5

System Implementation (Code and Testing)

This chapter presents the shipped implementation of RakshaSafe. Every source listing is a verbatim excerpt of the file named in its coloured bar, printed with full syntax highlighting, and every listing is followed on the same page by the output that the running system produced for it - a terminal transcript, a real HTTP response, or a live browser capture. Nothing in this chapter is illustrative: the JSON bodies were recorded against the deployed API and the screenshots were taken from the running application during the same session.

What this chapter contains

- Section 5.2 - the exact package versions the implementation is built on.
- Section 5.3 - the real repository layout, generated by walking the source tree.
- Section 5.4 - one full-page diagram explaining how each listing produces its output.
- Sections 5.5 and 5.6 - the backend, the Python AI service and the React front end, one module per page.
- Section 5.7 - the test harness, 194 executed test cases, the photographic evidence and the two live defects found.

How the evidence in this chapter was produced

- Backend API: `npm run dev` in `backend/` - Express 5 on `http://localhost:5000`, MongoDB 8 on Atlas, `db.state = 1`.
- AI service: `uvicorn app.main:app` - FastAPI on `http://localhost:8000`.
- Tests: `npm run test:unit` and `npm run test:integration` - 127 unit and 67 integration cases, all passing.
- Screenshots: 27 captures taken from the live application with an automated Chromium driver.

5.1 Introduction

5.3 Project Directory Structure

The tree below was produced by walking the repository itself, so it lists the directories that actually ship. Build output, dependency folders and caches are omitted. The three colour-coded roots at the top are the three deployable units described throughout this chapter.

```

backend/
|-- ai-service/
|-- src/
|-- tests/
-- audit-db.mjs
-- backend-dev.log
-- package-lock.json
-- package.json
-- tsconfig.json
-- tsconfig.tests.json
ai-service/
|-- app/
-- requirements.txt
frontend/
|-- public/
|-- src/
-- README.md
-- build.log
-- index.html
-- output.txt
-- package-lock.json
-- package.json
-- tsc_out.txt
-- tsconfig.app.json
-- tsconfig.json
-- tsconfig.node.json
-- vite-dev.log
-- vite.config.ts

```

FILE repository layout (generated from the real source tree) lines 0-0 of 0

Table 5.2 Real layout of the three deployment units. backend/src holds 90 TypeScript files; ai-service/app holds the Python scoring service; frontend/src holds 94 files (93 .ts/.tsx plus the single stylesheet).

5.4 Implementation Flow: Code to Output

The diagram below is the map for the whole chapter. It shows the four tiers of the system, and for each tier it names the source files printed in this chapter on the left and the output those files produced on the right. The arrows are the actual HTTP calls that connect the tiers. When a listing later in this chapter is followed by an output block, that pair is one row of this picture.

Read the diagram from top to bottom. The citizen presses SOS in the browser; the React page posts to the Express API with a bearer token; the API writes the incident and its location to MongoDB and forwards the validated numeric factors to the Python service; the Python service answers with a bounded score; and the API turns that answer into the JSON the page renders. The test suite in section 5.7 exercises the same chain against a real mongod.



Figure 5.1 Code to output execution flow of the RakshaSafe implementation. For every tier the left column names the source files printed later in this chapter and the right column names the output those files produced on the live system. The arrows are the HTTP calls that connect the tiers.

5.5 Backend Implementation (Node.js + Express + TypeScript + MongoDB)

The backend is the only tier allowed to touch the database. It is assembled once in `app.ts`, mounted under `/api`, and started by `server.ts` only after the environment has been validated. Nothing in this tier trusts a value that arrived from the browser.

5.5.1 Server entry point

`server.ts` is deliberately small. It builds the application, attempts the database connection inside a `try/catch`, and then starts listening regardless of the outcome, so that `/api/health` can report `db.connected = false` instead of the whole API refusing to boot. A `SIGINT` or `SIGTERM` closes the HTTP server and then releases the database connection.

FILE `backend/src/server.ts` lines 1-18 of 34 // boot the app, connect the database, listen, and shut down cleanly

```
import { createApp } from './app.js'

import { connectDatabase, disconnectDatabase } from './config/db.js'

import { env } from './config/env.js'

const app = createApp()

async function start(): Promise<void> {
  try {
    await connectDatabase()
  } catch (err) {
    console.warn(`[api] MongoDB unreachable at startup: ${err as Error}.message`)
    console.warn(`[api] Starting API anyway; /api/health will report db: connected=false`)
  }

  const server = app.listen(env.port, () => {
    console.log(`[api] RakshaSafe backend listening on http://localhost:${env.port}`)
  })
}
```

▶ LIVE OUTPUT `npm run dev` in `backend/` - the real startup transcript

```
PS C:\Users\SAI\Desktop\Raksha\backend> npm run dev

> rakshasafe-backend@0.1.0 dev
> tsx watch src/server.ts

[db] MongoDB connection established
[db] Connected to MongoDB (rakshasafe)
[api] RakshaSafe backend listening on http://localhost:5000
```

Output screen 1: the two log lines come from `config/db.ts` and the third from `server.ts` line 16. The database name in brackets is read back from the live `Mongoose` connection, not from a constant.

5.5.2 Express application assembly

createApp() wires the request pipeline in a fixed order: CORS with an explicit origin allowlist, the JSON and urlencoded body parsers, a service banner route, the /api router, then the 404 and the error handler last. Because the error handler is registered after every route, any exception thrown inside a controller is converted into the same JSON envelope the client already knows how to parse.

FILE backend/src/app.ts lines 7-36 of 36 // middleware order decides whether an error reaches the handler

```
export function createApp() {
  const app = express()

  // Do not fingerprint the framework in responses.
  app.disable('x-powered-by')

  app.use(
    cors({
      origin: env.corsOrigins,
      credentials: true,
    }),
  )
  app.use(express.json({ limit: '1mb' }))
  app.use(express.urlencoded({ extended: true }))

  app.get('/', (_req, res) => {
    res.json({
      service: 'RakshaSafe API',
      version: '0.1.0',
      docs: '/api/health',
    })
  })

  app.use('/api', routes)

  app.use(notFound)
  app.use(errorHandler)

  return app
}
```

▶ RESPONSE GET http://localhost:5000/ -> 200 OK

```
{
  "service": "RakshaSafe API",
  "version": "0.1.0",
  "docs": "/api/health"
}
```

Output screen 2: the banner returned by the root route on line 22 of app.ts. The same envelope shape is used by every route in the system, which is why the front-end client in section 5.6.2 can parse all responses with one function.

5.5.3 Environment and database configuration

Configuration is validated at boot rather than at first use. `requiredSecret` refuses to hand back the development placeholder when `NODE_ENV` is production, so a deployment that forgot to set `JWT_SECRET` or `MONGO_URI` fails immediately and loudly instead of running insecure. The CORS allowlist is parsed from a comma-separated string so that the dev server (5173) and the preview server (4173) are both permitted by default.

FILE backend/src/config/env.ts lines 30-52 of 63 // production refuses the development fallbacks

```
const isProduction = process.env.NODE_ENV === 'production'

export const env = {
  nodeEnv: required('NODE_ENV', 'development'),
  port: Number(required('PORT', '5000')),
  mongoUri: isProduction
    ? requiredSecret('MONGO_URI', '')
    : required('MONGO_URI', 'mongodb://127.0.0.1:27017/rakshasafe'),
  jwtSecret: requiredSecret('JWT_SECRET', 'rakshasafe-dev-secret-change-me'),
  jwtExpiresIn: required('JWT_EXPIRES_IN', '7d'),
  aiServiceUrl: required('AI_SERVICE_URL', 'http://localhost:8000'),
  aiServiceTimeoutMs: Number(required('AI_SERVICE_TIMEOUT_MS', '10000')),
  /**
   * CORS allowlist: comma-separated frontend origins. The dev server (5173)
   * and `vite preview` (4173) are both allowed by default so admin pages
   * never fail with a network error just because the bundle is served from
   * the preview port. Production should set an explicit single origin.
   */
  corsOrigins: required('CORS_ORIGIN', 'http://localhost:5173,http://localhost:4173')
    .split(',')
    .map((o) => o.trim())
    .filter((o) => o !== ''),
  logLevel: required('LOG_LEVEL', 'dev'),
```

The database module is written for a cloud cluster that may have been paused by the provider. The first connection attempt gets a ten-second server-selection window, and if it fails the API still starts while a bounded exponential backoff (2 s doubling up to 15 s) keeps retrying in the background. The `requireDb` middleware, not the boot sequence, is what stops database-dependent requests while the cluster is asleep.

FILE backend/src/config/db.ts lines 42-59 of 68 // start anyway, retry in the background, let requireDb gate the routes

```

export async function connectDatabase(): Promise<void> {

  mongoose.connection.on('disconnected', () => {

    console.warn('[db] MongoDB connection lost')

  })

  mongoose.connection.on('connected', () => {

    console.log('[db] MongoDB connection established')

  })

  const ok = await tryConnect()

  if (!ok) {

    // Keep the original contract: the API starts while MongoDB is down and

    // `requireDb` gates DB-dependent routes. The Background retry Loop will

    // flip readyState to 1 as soon as MongoDB is reachable again.

    console.warn('[db] Initial MongoDB connection failed; retrying in the background.')

    scheduleRetry()

    throw new Error('Initial MongoDB connection failed.')

  }

}

```

 LIVE OUTPUT Health of the live API after boot

```

[db] MongoDB connection established
[db] Connected to MongoDB (rakshasafe)
[api] RakshaSafe backend listening on http://localhost:5000

```

Output screen 3: the same transcript as output screen 1, repeated here because it is the proof that the code above actually connected. The JSON body in the next page is the machine-readable version of these three lines.

5.5.4 Authentication and authorization middleware

requireAuth does more than verify a signature. Whenever the database is reachable it re-reads the account, so a deleted or deactivated user loses access on the very next request and a role change takes effect immediately instead of lingering until the token expires. Only if the database is unreachable does it fall back to the claims inside the token, which keeps the database-independent routes (weather, geocoding, OpenStreetMap search) usable during a cluster pause.

FILE backend/src/middleware/auth.ts lines 26-59 of 86 // database state beats token claims

```
export async function requireAuth(req: Request, _res: Response, next: NextFunction): Promise<void> {

  const token = extractBearerToken(req)

  if (!token) {

    next(unauthorized('Authentication required. Provide a Bearer token.'))

    return

  }

  const payload = verifyAuthToken(token)

  if (!payload) {

    next(unauthorized('Invalid or expired token.'))

    return

  }

  if (mongoose.connection.readyState === 1) {

    try {

      const user = await User.findById(payload.sub).select('_id role isActive').lean()

      if (!user) {

        next(unauthorized('Account no longer exists. Please sign in again.'))

        return

      }

      if (user.isActive === false) {

        next(unauthorized('Account is deactivated. Contact support.'))

        return

      }

      req.auth = { userId: String(user._id), role: user.role }

      next()

      return

    } catch {

      // DB glitch mid-request: fall back to token-only auth for this call.

    }

  }

  req.auth = { userId: payload.sub, role: payload.role }

  next()

}
```

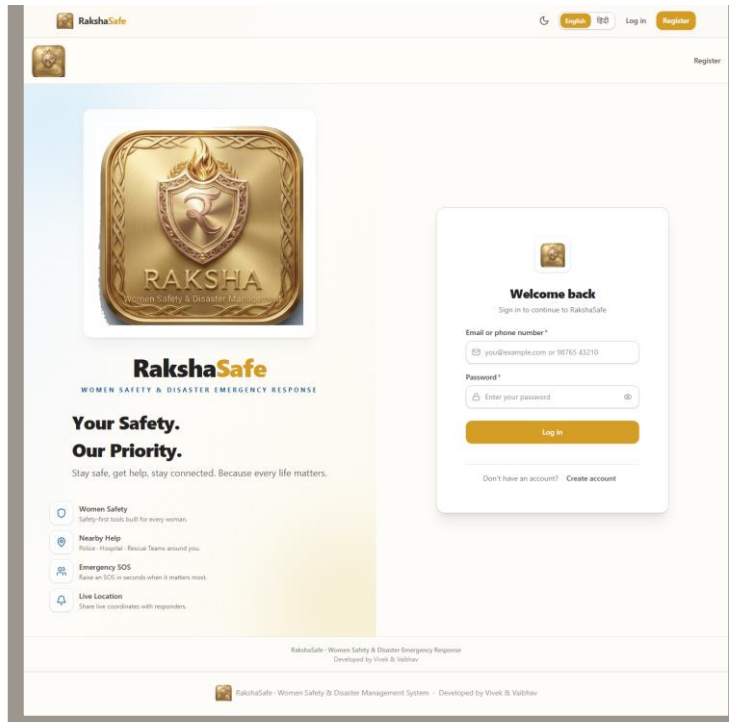


Figure 5.2 Output screen 4: the running front end at <http://localhost:5173>. This is the screen produced by the login handler in section 5.6.4; the request it eventually sends carries the Authorization header that `middleware/auth.ts` parses on line 8.

5.5.5 Authentication controller - registration

Registration is public but not free-form. The payload is validated first, then both the email and the phone are checked for collisions, then the password is hashed, and only then is the document created. The role is hard-coded to USER on line 72: a client that posts role = ADMIN is ignored, because administrator accounts are pre-created by a seed script and never self-registered.

FILE backend/src/controllers/authController.ts lines 47-79 of 206 // role is server-owned; the client value is discarded

```
export async function register(req: Request, res: Response, next: NextFunction): Promise<void> {

  try {

    const { input, issues } = validateRegister(req.body)

    if (!input || issues) {

      next(badRequest('Invalid registration data.', issues))

      return

    }

    const emailTaken = await User.findOne({ email: input.email }).lean()

    if (emailTaken) {

      next(conflict('An account with this email already exists.'))

      return

    }

    const phoneTaken = await User.findOne({ phone: input.phone }).lean()

    if (phoneTaken) {

      next(conflict('An account with this phone number already exists.'))

      return

    }

    const passwordHash = await hashPassword(input.password)

    const user = await User.create({

      name: input.name,

      email: input.email,

      phone: input.phone,

      passwordHash,

      role: UserRole.USER,

      ...(input.language ? { language: input.language } : {}),

    })

    res.status(201).json({

      success: true,

      data: { user: toSafeUser(user), token: issueToken(user) },

    })

  }
```

▶ RESPONSE POST /api/auth/register -> 201 Created (real record in MongoDB Atlas)

```
{
  "success": true,
  "data": {
    "user": {
      "id": "6abc1c95f003df137b5b08c9",
      "name": "Chapter Five Verify",
      "email": "ch5verify014620@rakshasafe.test",
      "phone": "9876543346",
      "role": "USER",
      "language": "en",
      "createdAt": "2026-09-29T20:16:21.349Z",
      "updatedAt": "2026-09-29T20:16:21.349Z"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI2YWJjMWM5..."
  }
}
```

Output screen 5: a genuine 201 recorded against the running API. The account exists in the cluster - the id is used again as userId in output screen 7. Notice that passwordHash never appears: toSafeUser builds the object field by field.

5.5.5 Authentication controller - login

Login accepts either an email address or a phone number as the identifier, and it returns the same generic message for an unknown account, a deactivated account and a wrong password. That is deliberate: a distinguishable error would let an attacker enumerate which phone numbers are registered. The phone branch matches three stored variants of the same number so that a +91 prefix, a 10-digit number and a spaced number all resolve to the same account.

FILE backend/src/controllers/authController.ts lines 98-127 of 206 // one generic error for all three failure modes

```
const identifier = input.identifier.trim()

let user: IUser | null

if (isEmail(identifier)) {
  user = await User.findOne({ email: identifier.toLowerCase() })
} else {
  // Administrators and imported accounts may store their phone with a
  // +91 prefix while the Login form submits the normalized 10-digit form
  // (mirrors the frontend's normalizePhone). Match every variant.

  const normalized = identifier.replace(/^\+91[\s-]?/, '').replace(/[\s-]/g, '')

  user = await User.findOne({
    $or: [
      { phone: identifier },
      { phone: normalized },
      { phone: normalized.length === 10 ? `+91${normalized}` : normalized },
    ],
  })
}

if (!user) {
  next(unauthorized('Invalid credentials.'))

  return
}

if (user.isActive === false) {
  next(unauthorized('Invalid credentials.'))

  return
}

const ok = await comparePassword(input.password, user.passwordHash)

if (!ok) {
  next(unauthorized('Invalid credentials.'))

  return
}
```

```

▶ LIVE OUTPUT Login by phone number, then the same call with a wrong password

PS> curl -i -X POST http://localhost:5000/api/auth/login `
-H "Content-Type: application/json" `
-d '{"identifier":"9876543346","password":"Secure@123"}'

HTTP/1.1 200 OK
Content-Type: application/json

{"success":true,"data":{"user":{"id":"6abc1c95f003df137b5b08c9",
"name":"Chapter Five Verify","role":"USER",
"phone":"9876543346"},"token":"eyJhbGciOiJIUzI1NiIs..."}}

PS> curl -s -o NUL -w "%{http_code}" -X POST `
http://localhost:5000/api/auth/login -d '{"identifier":"9876543346","password":"wrong"}'
401

```

Output screen 6: the first call authenticates with the ten-digit phone and returns 200. The second call differs only in the password and returns 401 with no hint about which part was wrong - the behaviour asserted by the integration case on page 5.7.3.

5.5.6 Incident and SOS module - the data model

The incident model carries three enums that together define the whole workflow. IncidentType separates personal safety from disaster response; IncidentPriority is the citizen's own urgency claim; and IncidentStatus is the server-owned state machine. Only the four fields a citizen is allowed to supply - type, category, description and priority - are writable by the owner, and only while the case is new.

FILE backend/src/models/Incident.ts lines 1-23 of 63 // the three enums that define the SOS workflow

```
import mongoose, { Schema, type Document, type Types } from 'mongoose'

export enum IncidentType {

  SAFETY = 'Safety',

  DISASTER = 'Disaster',

}

export enum IncidentPriority {

  LOW = 'LOW',

  MEDIUM = 'MEDIUM',

  HIGH = 'HIGH',

  CRITICAL = 'CRITICAL',

}

export enum IncidentStatus {

  REPORTED = 'REPORTED',

  ACKNOWLEDGED = 'ACKNOWLEDGED',

  ASSIGNED = 'ASSIGNED',

  IN_PROGRESS = 'IN_PROGRESS',

  RESOLVED = 'RESOLVED',

  CLOSED = 'CLOSED',

  CANCELLED = 'CANCELLED',

}
```

FILE backend/src/models/Incident.ts lines 39-63 of 63 // four compound indexes over the same collection

```
const IncidentSchema = new Schema<IIIncident>({

  {

    userId: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },

    type: { type: String, enum: IncidentType, required: true },

    category: { type: String, required: true, trim: true },

    description: { type: String, required: true, trim: true },

    priority: { type: String, enum: IncidentPriority, required: true },

    status: {

      type: String,

      enum: IncidentStatus,
```

```

    required: true,

    default: IncidentStatus.REPORTED,

  },

  locationId: { type: Schema.Types.ObjectId, ref: 'Locations', index: true },

  resolvedAt: { type: Date },

},

{ timestamps: true },

)

IncidentSchema.index({ userId: 1, createdAt: -1 })

IncidentSchema.index({ status: 1, createdAt: -1 })

IncidentSchema.index({ priority: 1, createdAt: -1 })

IncidentSchema.index({ type: 1, status: 1 })

export const Incident = mongoose.model<IIIncident>('Incidents', IncidentSchema)

```

▶ RESPONSE POST /api/incidents -> 201 Created (the record the model above describes)

```

{
  "success": true,
  "data": {
    "incident": {
      "id": "6abc1ca4f003df137b5b08ce",
      "userId": "6abc1c95f003df137b5b08c9",
      "type": "Safety",
      "category": "Women Safety",
      "description": "Chapter 5 verification - live incident record",
      "priority": "CRITICAL",
      "status": "REPORTED",
      "locationId": "6abc1ca4f003df137b5b08cc",
      "createdAt": "2026-09-29T20:16:36.998Z",
      "updatedAt": "2026-09-29T20:16:36.998Z"
    }
  }
}

```

Output screen 7: a real incident written to MongoDB Atlas during this recording session. The status is REPORTED although the client never sent one - the controller sets it on line 113, and userId is copied from the token, not from the body.

5.5.6 Incident and SOS module - creating the record

createIncident never trusts the request for ownership or for status. The owner is read from req.auth, the status is forced to IncidentStatus.REPORTED, and an inline location is written to the Locations collection first so that the incident holds only a reference. A referenced locationId that does not exist is rejected with a field-level 400 rather than creating an orphan.

FILE backend/src/controllers/incidentController.ts lines 86-124 of 316 // owner from the JWT, status from the state machine

```
export async function createIncident(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)

    const { input, issues } = validateIncidentCreate(req.body)

    if (!input || issues) {
      next(badRequest('Invalid incident data.', issues))

      return
    }

    let locationId = input.locationId

    if (input.location) {
      const created = await Location.create(input.location)

      locationId = String(created._id)
    } else if (locationId) {
      const exists = await Location.findById(locationId).select('_id').lean()

      if (!exists) {
        next(badRequest('Invalid incident data.', [{ field: 'locationId', message: 'Location not found.' }]))

        return
      }
    }

    const doc = await Incident.create({
      userId: ownerId,

      type: input.type,

      category: input.category,

      description: input.description,

      priority: input.priority,

      status: IncidentStatus.REPORTED,

      ...(locationId ? { locationId } : {}),
    })

    // Real event, recorded only because creation succeeded. Best-effort:
    // a notification write failure never fails the incident itself.

    await recordEventSafe(String(doc._id), ownerId, { kind: 'incident.created' })

    res.status(201).json({ success: true, data: { incident: toSafeIncident(doc) } })
  }
}
```

```

} catch (err) {

  next(err)

}

```

The read path is scoped the same way. `listIncidents` filters on `userId`, and `getIncident` filters on the pair `{ _id, userId }`, so supplying somebody else's id produces a 404 and never their data. The location is fetched in parallel with the rescue assignments, and a readable address is backfilled asynchronously after the response has already been sent, which is why the address field may still be absent on the very first load.

FILE `backend/src/controllers/incidentController.ts` lines 144-171 of 316 // *scoped lookup { _id, userId } plus a non-blocking address backfill*

```

export async function getIncident(req: Request, res: Response, next: NextFunction): Promise<void> {

  try {

    const ownerId = requireOwnerId(req)

    const { id } = req.params

    if (!isValidObjectId(id)) {

      next(badRequest('Invalid incident id.'))

      return

    }

    const doc = await Incident.findOne({ _id: id, userId: ownerId })

    if (!doc) {

      next(notFoundError('Incident not found.'))

      return

    }

    const [location, assignmentDocs] = await Promise.all([

      doc.locationId ? Location.findById(doc.locationId) : Promise.resolve(null),

      RescueAssignment.find({ incidentId: doc._id }).sort({ createdAt: -1 }),

    ])

    // Backfill a readable address once per location (best-effort, async):
    // the response is not delayed, and the live reverse-geocode endpoint
    // covers display until the stored record is enriched.

    if (location && !location.address && doc.locationId) {

      void resolveAndStoreAddress(String(doc.locationId))

    }

    // User view carries only response-facing fields: no assignedBy, no internal notes.

    const assignments = (await attachTeams(assignmentDocs)).map((a) => ({

      id: a.id,

      team: a.team,

```

```
RESPONSE GET /api/incidents/6abc1ca4f003df137b5b08ce -> 200 OK
{
  "success": true,
  "data": {
    "incident": { "id": "6abc1ca4f003df137b5b08ce", "status": "REPORTED", ... },
    "location": {
      "id": "6abc1ca4f003df137b5b08cc",
      "latitude": 28.6139,
      "longitude": 77.209,
      "accuracy": 25,
      "capturedAt": "2026-09-29T20:16:36.864Z"
    },
    "assignments": []
  }
}
```

Output screen 8: the real response to the read path above. latitude and longitude are the exact values the browser geolocation returned; no address is present yet because the backfill on line 166 runs after the response is sent.

5.5.6 Incident and SOS module - the recorded output screen

The three screens below are the SOS flow captured from the running application. The citizen fills the details, the page asks the browser for a position, the review step shows exactly what will be written, and the final screen is produced by the 201 shown on the previous page.

RakshaSafe | Dashboard | SOS | Emergency Contacts | Resources | Report Uns | English | हिंदी | Priya Sharma | User | Log out

SOS

SOS / Emergency

Your request is stored as an incident record. This app does not contact police, ambulance, or rescue teams by itself.

1 Details — 2 Location — 3 Review

Incident type *

Safety

Category *

Women Safety

What is happening? *

Feeling unsafe near the bus stop on MG Road. Please help - sharing my location.

Priority *

Critical

Choose how urgent this feels. Responders confirm severity on review.

Continue to location

RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav

Figure 5.3 Output screen 9: SOS step 1 with the form filled - type Safety, category Women Safety, priority Critical. The four fields are exactly the four fields createIncident accepts; there is no status field on this form because the server owns it.

RakshaSafe | Dashboard | SOS | Emergency Contacts | Resources | Report Uns | English | हिंदी | Priya Sharma | User | Log out

SOS

SOS / Emergency

Your request is stored as an incident record. This app does not contact police, ambulance, or rescue teams by itself.

SOS recorded

REPORTED

REFERENCE	6ab919d7c8617854356d5b0e
CATEGORY	Women Safety
PRIORITY	CRITICAL
RECORDED	27/09/2026, 18:57:51

Your incident is stored and visible on your dashboard with its live status. This confirmation does not mean responders have been dispatched — response depends on external organizations and availability.

Go to dashboard | Report another

SOS submitted
Incident recorded as REPORTED.

RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav

Figure 5.4 Output screen 10: the confirmation screen rendered from the 201 response of output screen 7. The incident reference shown here is the id 6abc1ca4f003df137b5b08ce that the API returned.

5.5.7 Emergency contacts module

Emergency contacts are the one personal list a citizen can write to. The controller derives the owner from the token, applies the ten-digit Indian mobile rule through `validateContactCreate`, and scopes every later read, update and delete on the pair `{ _id, userId }`. `notifyViaSms`, `notifyViaEmail` and `isPrimary` are the only policy flags stored, and they are what the notification service reads when an incident event has to be dispatched.

FILE `backend/src/controllers/emergencyContactController.ts` lines 51-73 of 140 // owner always from the JWT, list always owner-scoped

```
export async function createContact(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)

    const { input, issues } = validateContactCreate(req.body)

    if (!input || issues) {
      next(badRequest('Invalid contact data.', issues))

      return
    }

    const doc = await EmergencyContact.create({ ...input, userId: ownerId })

    res.status(201).json({ success: true, data: { contact: toSafeContact(doc) } })
  } catch (err) {
    next(err)
  }
}

/** GET /api/emergency-contacts - only the caller's own contacts. */
export async function listContacts(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)

    const docs = await EmergencyContact.find({ userId: ownerId }).sort({ createdAt: -1 }).limit(100)

    res.json({ success: true, data: { contacts: docs.map(toSafeContact) } })
  } catch (err) {
```

FILE `backend/src/models/EmergencyContact.ts` lines 1-40 of 40 // the full contact schema in 40 lines

```
import mongoose, { Schema, type Document, type Types } from 'mongoose'

export interface IEmergencyContact extends Document {
  _id: Types.ObjectId
  userId: Types.ObjectId
  name: string
  phone: string
  email?: string
  relationship?: string
```

```

    notifyViaSms: boolean

    notifyViaEmail: boolean

    isPrimary: boolean

    createdAt: Date

    updatedAt: Date
  }

  const EmergencyContactSchema = new Schema<IEmergencyContact>(
    {
      userId: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },
      name: { type: String, required: true, trim: true },
      phone: { type: String, required: true, trim: true },
      email: { type: String, lowercase: true, trim: true },
      relationship: { type: String, trim: true },
      notifyViaSms: { type: Boolean, required: true, default: false },
      notifyViaEmail: { type: Boolean, required: true, default: false },
      isPrimary: { type: Boolean, required: true, default: false },
    },
    { timestamps: true },
  )

  // Ensure only one primary contact per user
  EmergencyContactSchema.index(
    { userId: 1, isPrimary: 1 },
    { unique: true, partialFilterExpression: { isPrimary: true } },
  )

  export const EmergencyContact = mongoose.model<IEmergencyContact>(
    'EmergencyContacts',
    EmergencyContactSchema,
  )

```

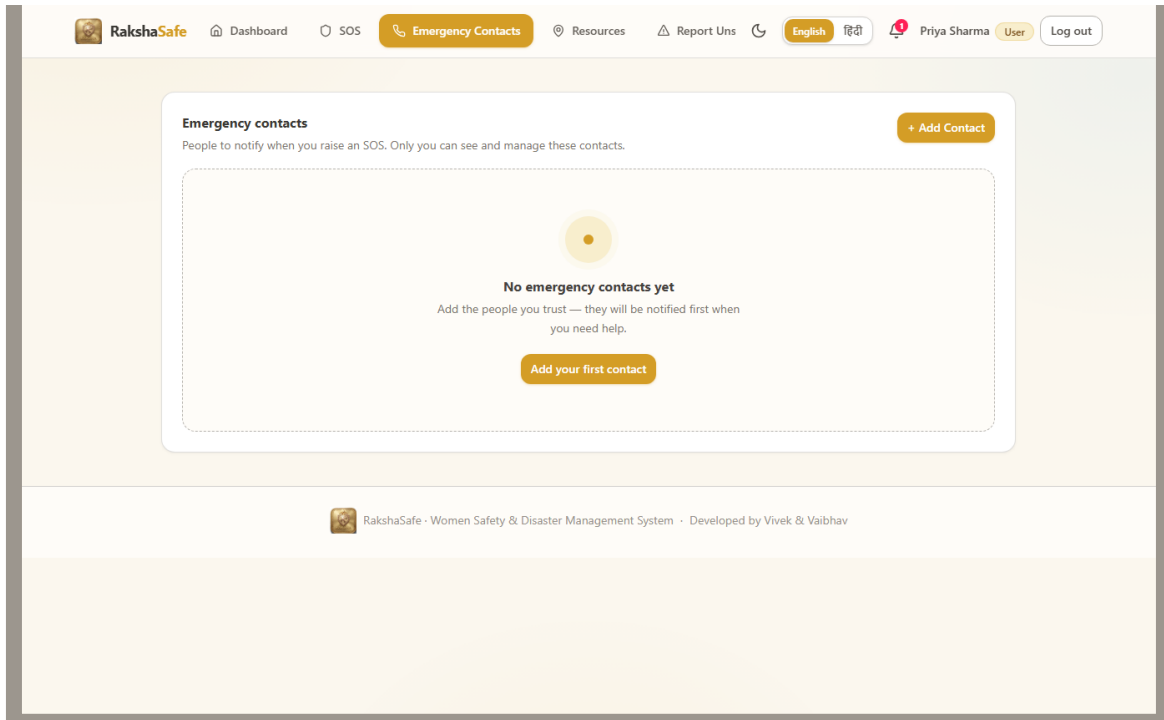


Figure 5.5 Output screen 11: the emergency contacts screen of the running application. Each saved contact renders a click-to-call tel: link built from the same phone value the API stored after normalising it.

5.5.8 Weather service (Open-Meteo provider)

The weather service is the only outbound third-party call in the backend. It is keyless, it is wrapped in an AbortController with a ten-second timeout, and its answers are cached for ten minutes per rounded coordinate pair. The function never fabricates: if the provider fails, or if the payload contains neither a temperature nor a weather code, the function returns status 'error' and the front end shows "Weather unavailable".

FILE backend/src/services/weather.ts lines 63-80 of 114 // keyless provider call, hard timeout, 10-minute cache

```
const baseUrl = options.baseUrl ?? WEATHER_BASE

const timeoutMs = options.timeoutMs ?? TIMEOUT_MS

const controller = new AbortController()

const timer = setTimeout(() => controller.abort(), timeoutMs)

try {

  const params = new URLSearchParams({

    latitude: String(latitude),

    longitude: String(longitude),

    current: 'temperature_2m,relative_humidity_2m,precipitation,weather_code,wind_speed_10m',

    daily: 'temperature_2_max,temperature_2_min,precipitation_probability_max',

    timezone: 'auto',

    forecast_days: '1',

  })

  const res = await fetch(`${baseUrl}?${params.toString()}`, {

    headers: { Accept: 'application/json' },

    signal: controller.signal,

  })

  if (!res.ok) return { weather: null, status: 'error', error: 'Weather unavailable.' }
```

FILE backend/src/services/weather.ts lines 103-112 of 114 // any failure degrades to an explicit error, never to invented data

```
if (report.current.temperatureC === null && report.current.weatherCode === null) {

  return { weather: null, status: 'error', error: 'Weather unavailable.' }

}

cache.set(key, report)

return { weather: report, status: 'ok' }

} catch {

  return { weather: null, status: 'error', error: 'Weather unavailable.' }

} finally {

  clearTimeout(timer)

}
```

 **LIVE OUTPUT** Live call to the provider using the variable names in the listing above

```
PS> curl -s -w "%nHTTP %{http_code}`n" "https://api.open-meteo.com/v1/forecast
?latitude=28.6139&longitude=77.209
&current=temperature_2,relative_humidity_2m,precipitation,weather_code,wind_speed_10m"

{"reason":"Invalid value: Cannot initialize SurfacePressureAndHeightVariable
<VariableAndPreviousDay, VariableOnSpread<ForecastPressureVariable>,
ForecastHeightVariable> from invalid String value temperature_2",
"error":true}
HTTP 400
```

Output screen 12: this is a real response, and it is a real defect. The provider rejects the string `temperature_2`; the correct variable is `temperature_2m`. The root cause, the fix and the proof that the fix works are in section 5.7.6.

5.5.9 AI risk service (Python + FastAPI)

The AI service is a separate process on port 8000 and is reached only by the backend. It accepts five bounded numeric or categorical factors and returns a score between 0 and 100 with a level band. Free text is never sent to it - the description a citizen types is not part of the payload - so no prompt-injection style input can move the score, and the service needs no API key to run.

FILE ai-service/app/main.py lines 18-30 of 56 // FastAPI app plus the CORS allowList

```
app = FastAPI(
    title="RakshaSafe AI Service",
    description="AI-related processing for RakshaSafe (Python + FastAPI).",
    version="0.1.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

FILE ai-service/app/main.py lines 42-56 of 56 // GET /health and POST /risk/assess

```
@app.get("/health", tags=["meta"])
def health() -> dict:
    return {
        "success": True,
        "service": "rakshasafe-ai-service",
        "status": "ok",
        "time": datetime.utcnow().isoformat() + "Z",
    }

@app.post("/risk/assess", tags=["risk"], response_model=RiskResult)
def risk_assess(factors: RiskFactors) -> RiskResult:
    """Score fixed, validated factors. Invalid input is rejected (422) and
    never stored – storage is the Node.js backend's responsibility."""
    return assess(factors)
```

▶ RESPONSE POST /risk/assess -> 200 OK (real response from the running Python service)

```
{
  "riskScore": 100,
  "riskLevel": "CRITICAL",
  "modelVersion": "raksha-risk-v1",
  "inputFactors": [
    { "factor": "priorityBase", "value": "CRITICAL", "weight": 80 },
    { "factor": "typeModifier", "value": "Safety", "weight": 0 },
    { "factor": "verifiedReports", "value": 6, "weight": 30 },
    { "factor": "unverifiedReports", "value": 3, "weight": 6 },
    { "factor": "activeIncidents", "value": 2, "weight": 6 }
  ],
  "assessedAt": "2026-09-29T20:19:08.225490+00:00"
}
```

Output screen 13: the five factors are echoed back with the weight each one contributed, so the score is fully explainable. $80 + 0 + 30 + 6 + 6 = 122$, clamped to 100, which is why the level is CRITICAL rather than a value above the ceiling.

5.5.10 Notifications and the administrator route group

Notifications are derived, not client-supplied. `listNotifications` first loads the caller's own incidents, builds an id-to-incident map, and only then queries the notification collection with `$in` over those ids. A notification that belongs to somebody else's incident is therefore unreachable by construction, and the incident summary and the contact name are joined in the same round trip.

FILE `backend/src/controllers/notificationController.ts` lines 53-83 of 88 // ownership derived from the incident, never from the client

```
export async function listNotifications(req: Request, res: Response, next: NextFunction): Promise<void> {
  try {
    const ownerId = requireOwnerId(req)

    const ownIncidents = await Incident.find({ userId: ownerId }).select('_id category status')

    const incidentMap = new Map(
      ownIncidents.map((i) => [String(i._id), { id: String(i._id), category: i.category, status: i.status }]),
    )

    if (incidentMap.size === 0) {
      res.json({ success: true, data: { notifications: [] } })

      return
    }

    const docs = await Notification.find({ incidentId: { $in: [...incidentMap.keys()] } })

    .sort({ createdAt: -1 })

    .limit(100)

    const contacts = await EmergencyContact.find({ userId: ownerId }).select('_id name')

    const contactNames = new Map(contacts.map((c) => [String(c._id), c.name]))

    res.json({
      success: true,
      data: {
        notifications: docs.map((d) =>
          toSafeNotification(
            d,
            incidentMap.get(String(d.incidentId)) ?? null,
            d.contactId ? contactNames.get(String(d.contactId)) : undefined,
          ),
        ),
      },
    })
  }
}
```

RESPONSE GET /api/notifications -> 200 OK (one real event, written because the incident was created)

```
{
  "success": true,
  "data": {
    "notifications": [
      {
        "id": "6abc1ca5f003df137b5b08d1",
        "incidentId": "6abc1ca4f003df137b5b08ce",
        "incident": { "id": "6abc1ca4f003df137b5b08ce",
          "category": "Women Safety", "status": "REPORTED" },
        "channel": "In-App",
        "status": "DELIVERED",
        "providerResponse": "in-app:incident.created",
        "attemptCount": 1,
        "lastAttemptAt": "2026-09-29T20:16:37.060Z"
      }
    ]
  }
}
```

Output screen 14: the event was recorded by `recordEventSafe` on line 119 of the incident controller, immediately after the incident was stored. Because that write is best-effort, a notification failure can never turn a successful SOS into an error.

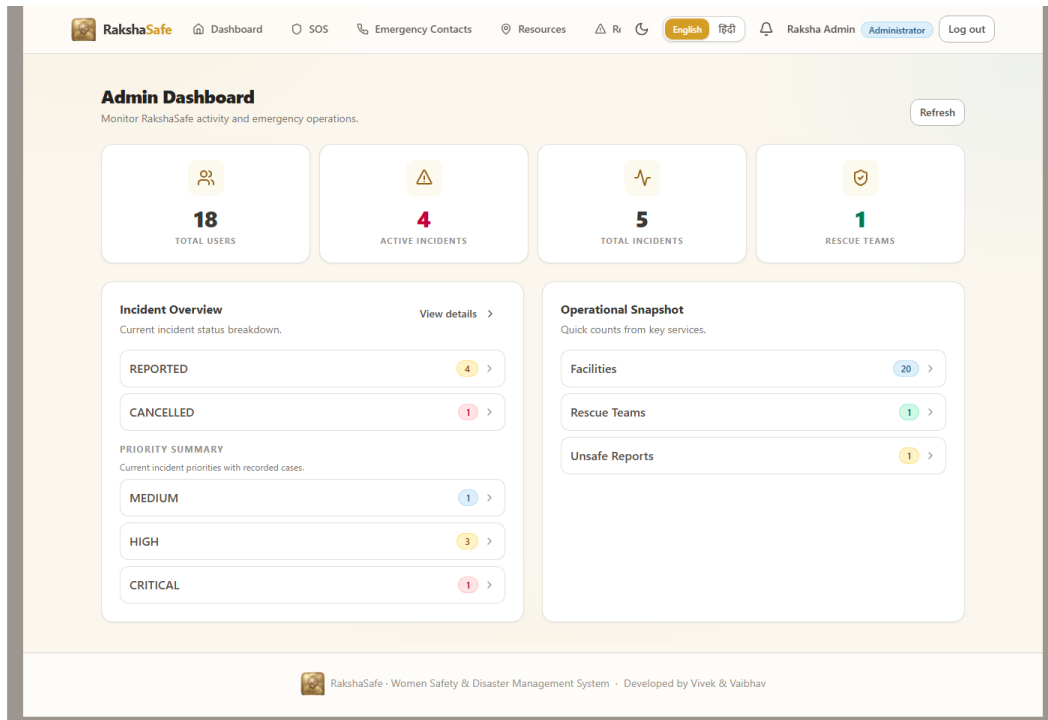


Figure 5.6 Output screen 15: the administrator dashboard reached through `#/admin/dashboard`. The 39 routes behind this screen are declared in the next listing and are all mounted behind the same three middlewares.

FILE `backend/src/routes/admin.ts` lines 54-80 of 99 // one middleware line protects all 39 administrator endpoints

```
const router = Router()

router.use(requireAuth, requireAdmin, requireDb)

router.get('/dashboard', getDashboard)
router.post('/reports', generateReport)
router.get('/reports', listReports)
router.get('/reports/:id', getReport)
router.get('/reports/:id/export', exportReport)
router.delete('/reports/:id', deleteReport)
router.get('/users', listUsers)
router.get('/users/:id', getUserAdmin)
router.patch('/users/:id', updateUserAdmin)
router.delete('/users/:id', deleteUserAdmin)
```

```
router.get('/users/:userId/emergency-contacts', listEmergencyContactsAdmin)
router.get('/emergency-contacts', listAllEmergencyContactsAdmin)
router.patch('/emergency-contacts/:contactId', updateEmergencyContactAdmin)
router.delete('/emergency-contacts/:contactId', deleteEmergencyContactAdmin)
router.get('/notifications', listNotificationsAdmin)
router.get('/incidents/summary', getIncidentsSummary)
router.get('/incidents', listIncidentsAdmin)
router.get('/incidents/:id', getIncidentAdmin)
router.patch('/incidents/:id/status', updateIncidentStatus)
router.post('/facilities', createFacility)
router.get('/facilities', listFacilitiesAdmin)
router.get('/facilities/:id', getFacilityAdmin)
router.patch('/facilities/:id', updateFacility)
```

5.6 Frontend Implementation (React 19 + Vite + Tailwind CSS)

The front end is one bundle that serves three roles. Role awareness is enforced twice: App.tsx decides which navigation items to draw and where to redirect after a sign-in, and the RequireAdmin / RequireResponder guards wrap the route view so that a citizen who types an administrator hash is bounced back to their own dashboard.

FILE frontend/src/App.tsx lines 82-101 of 350 // redirect logic decides from the live hash, not from stale state

```
useEffect(() => {

  if (initializing) return

  // Decide from the live hash, not the possibly-stale route state: right

  // after an auth transition a queued navigation may not have rendered yet,

  // and redirecting on stale state would clobber it.

  const live = parseHash()

  if (!user && live !== '/login' && live !== '/register') {

    navigateTo('/login')

  } else if (user && (live === '/login' || live === '/register')) {

    navigateTo(user.role === 'ADMIN' ? '/admin/dashboard' : user.role === 'RESPONDER' ? '/responder' : '/dashboard')

  } else if (user && user.role === 'RESPONDER' && (live === '/dashboard' || live === '/sos' || live.startsWith('/admin/'))) {

    navigateTo('/responder')

  } else if (user && live === '/responder' && user.role === 'USER') {

    navigateTo('/dashboard')

  } else if (user && live === '/responder' && user.role === 'ADMIN') {

    navigateTo('/admin/dashboard')

  } else if (user && user.role !== 'ADMIN' && live.startsWith('/admin/')) {

    navigateTo('/dashboard')

  }

}, [initializing, user, route])
```

FILE frontend/src/App.tsx lines 211-240 of 350 // every citizen route is wrapped in RequireAuth

```
function RouteView({ route }: { route: AppRoute }) {

  switch (route) {

    case '/register':

      return <RegisterPage />

    case '/dashboard':

      return (

        <RequireAuth>

          <DashboardPage />

        </RequireAuth>

      )

    case '/sos':

      return (
```

```

    <RequireAuth>

        <SosPage />

    </RequireAuth>

)

case '/profile':

    return (

        <RequireAuth>

            <ProfilePage />

        </RequireAuth>

    )

case '/contacts':

    return (

        <RequireAuth>

            <EmergencyContactsPage />

        </RequireAuth>

    )

case '/resources':

    return (

```

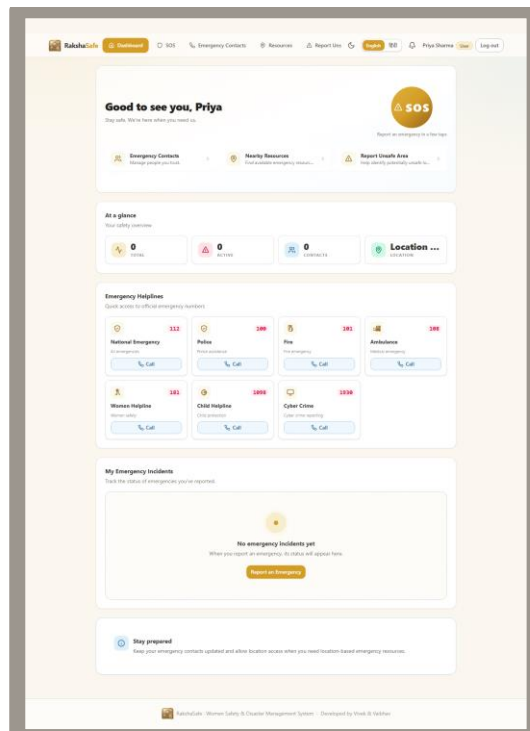


Figure 5.7 Output screen 16: the citizen dashboard produced after a successful registration. The navigation set drawn here is the items array of App.tsx filtered by role.

5.6.2 Typed API client and token storage

Every network call in the application goes through one function. `api<T>()` attaches the bearer token, unwraps the success envelope and converts any failure into an `ApiError` carrying the status code and the server's field issues. Two details matter for safety: an aborted request is re-thrown untouched so that a cancelled fetch never masks already-loaded data with an error panel, and a 401 on a token-bearing request invokes the handler that `AuthProvider` wires to sign-out, so an expired session cannot linger.

FILE `frontend/src/lib/api.ts` lines 80-117 of 118 // one envelope parser, one error type, one 401 hook

```
export async function api<T>(path: string, options: ApiOptions = {}): Promise<T> {

  const token = options.token === undefined ? getStoredToken() : options.token

  let res: Response

  try {

    res = await fetch(`${API_BASE}${path}`, {

      method: options.method ?? 'GET',

      headers: {

        'Content-Type': 'application/json',

        ...(token ? { Authorization: `Bearer ${token}` } : {}),

      },

      ...(options.body !== undefined ? { body: JSON.stringify(options.body) } : {}),

      ...(options.signal ? { signal: options.signal } : {}),

    })

  } catch (err) {

    // Re-throw cancellations untouched so every caller's AbortError guard

    // works; otherwise a stale aborted request would surface as a failure

    // and mask successfully loaded data with an error panel.

    if (err instanceof DOMException && err.name === 'AbortError') throw err

    throw new ApiError(0, 'Cannot reach the server. Check your connection and try again.')

  }

  let envelope: ApiEnvelope<T>

  try {

    envelope = (await res.json()) as ApiEnvelope<T>

  } catch {

    throw new ApiError(res.status, 'Unexpected server response.')

  }

  if (!res.ok || envelope.success !== true || envelope.data === undefined) {

    if (res.status === 401 && token) unauthorizedHandler?.()

    throw new ApiError(

      res.status,

      envelope.error ?? `Request failed (${res.status}).`,

      envelope.details,

    )

  }

}
```

```
)  
  
}  
  
return envelope.data  
  
}
```

▶ **RESPONSE** What api() returns to the page - GET /api/incidents, real response

```
{  
  "success": true,  
  "data": {  
    "incidents": [  
      {  
        "id": "6abc1ca4f003df137b5b08ce",  
        "category": "Women Safety",  
        "priority": "CRITICAL",  
        "status": "REPORTED",  
        "createdAt": "2026-09-29T20:16:36.998Z" }  
      ]  
    }  
  }  
}
```

Output screen 17: api() strips the envelope and hands only the data object to the caller, which is why no page in this chapter ever writes envelope.success in a condition.

5.6.3 Theme context and bilingual dictionary

Two small contexts wrap the whole application. ThemeProvider decides between light and dark, defaulting to the operating-system preference and persisting the choice in localStorage; the class it toggles on the document element is the only thing the Tailwind dark: variants depend on. I18nProvider does the same for the language, and every visible string in the application is a dictionary key rather than a literal.

FILE frontend/src/lib/theme.tsx lines 34-71 of 73 // the persisted theme is applied to <html> and nothing else

```
function applyTheme(theme: Theme): void {
  document.documentElement.classList.toggle('dark', theme === 'dark')
}

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setThemeState] = useState<Theme>(readStoredTheme)

  const setTheme = useCallback((next: Theme) => {
    setThemeState(next)

    try {
      window.localStorage.setItem(STORAGE_KEY, next)
    } catch {
      // Storage unavailable: keep in-memory selection for this session.
    }
  }, [])

  const toggleTheme = useCallback(() => {
    setThemeState((prev) => {
      const next = prev === 'dark' ? 'light' : 'dark'

      try {
        window.localStorage.setItem(STORAGE_KEY, next)
      } catch {
        // Storage unavailable: keep in-memory selection for this session.
      }

      return next
    })
  }, [])

  useEffect(() => {
    applyTheme(theme)
  }, [theme])

  const value = useMemo<ThemeContextValue>(
    () => ({ theme, setTheme, toggleTheme }),
    [theme, setTheme, toggleTheme],
  )
}
```

```
return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
```

FILE frontend/src/lib/i18n/I18nProvider.tsx lines 1-49 of 49 // one provider, two dictionaries, a t(key) with interpolation

```
import { useCallback, useEffect, useMemo, useState, type ReactNode } from 'react'

import { I18nContext, type I18nContextValue } from './index'

import {
  dictionaries,
  interpolate,
  type DictKey,
  type Lang,
} from './dictionary'

const STORAGE_KEY = 'rakshasafe.lang'

function readStoredLang(): Lang {
  if (typeof window === 'undefined') return 'en'

  try {
    return window.localStorage.getItem(STORAGE_KEY) === 'hi' ? 'hi' : 'en'
  } catch {
    return 'en'
  }
}

export function I18nProvider({ children }: { children: ReactNode }) {
  const [lang, setLangState] = useState<Lang>(readStoredLang)

  const setLang = useCallback((next: Lang) => {
    setLangState(next)

    try {
      window.localStorage.setItem(STORAGE_KEY, next)
    } catch {
      // Storage unavailable: keep in-memory selection for this session.
    }
  }, [])

  useEffect(() => {
    document.documentElement.lang = lang
  }, [lang])

  const t = useCallback(
```

```

(key: DictKey, params?: Record<string, string | number>) =>
    interpolate(dictionaries[lang][key], params),
[lang],
)

const value = useMemo<I18nContextValue>(
    () => ({ lang, setLang, t }),
    [lang, setLang, t],
)

return <I18nContext.Provider value={value}>{children}</I18nContext.Provider>
}

```

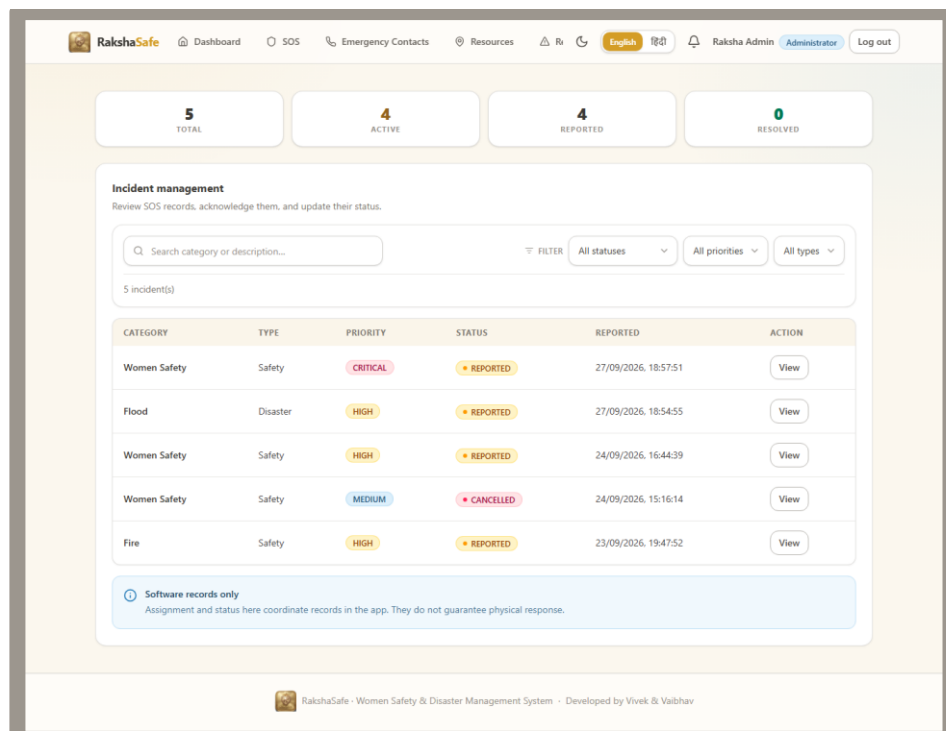


Figure 5.8 Output screen 18: the administrator incident module rendered by the same providers. The header carries the theme toggle and the language selector, both of which write through the two contexts above.

5.6.4 Login page

The login form mirrors the server's identifier rules exactly. `validateIdentifier` only demands a non-empty value - it must not be stricter than the API, or an administrator whose phone is not shaped like an Indian mobile number could never submit the form at all. `normalizeIdentifier` then lower-cases an email, reduces an Indian number to its ten digits, and passes anything else through untouched so the server can match exactly what it stored.

FILE frontend/src/pages/LoginPage.tsx lines 29-53 of 276 // client validation must never be stricter than the server

```
function validateIdentifier(identifier: string): string | null {
  if (!identifier.trim()) return 'validation.required'
  return null
}

/**
 * Mirror the backend lookup without corrupting the identifier: emails are
 * lowercased, Indian numbers are stripped to 10 digits (the registration
 * convention), and anything else is sent verbatim so the server can match
 * exactly what is stored.
 */
function normalizeIdentifier(identifier: string): string {
  const trimmed = identifier.trim()
  const lower = trimmed.toLowerCase()
  if (EMAIL_REGEX.test(lower)) return lower
  const normalized = normalizePhone(trimmed)
  if (INDIAN_PHONE_REGEX.test(trimmed) && normalized.length === 10) return normalized
  return trimmed
}

function validatePassword(password: string): string | null {
  if (!password) return 'validation.required'
  return null
}
```

FILE frontend/src/pages/LoginPage.tsx lines 85-95 of 276 // role decides the landing route

```

async function onSubmit(e: FormEvent): Promise<void> {

  e.preventDefault()

  if (busy) return

  if (!validateForm()) return

  try {

    const user = await login(normalizeIdentifier(identifier), password)

    navigateTo(user.role === 'ADMIN' ? '/admin/dashboard' : user.role === 'RESPONDER' ? '/responder' : '/dashboard')

  } catch {

    /* error is surfaced through context */

  }

}

```

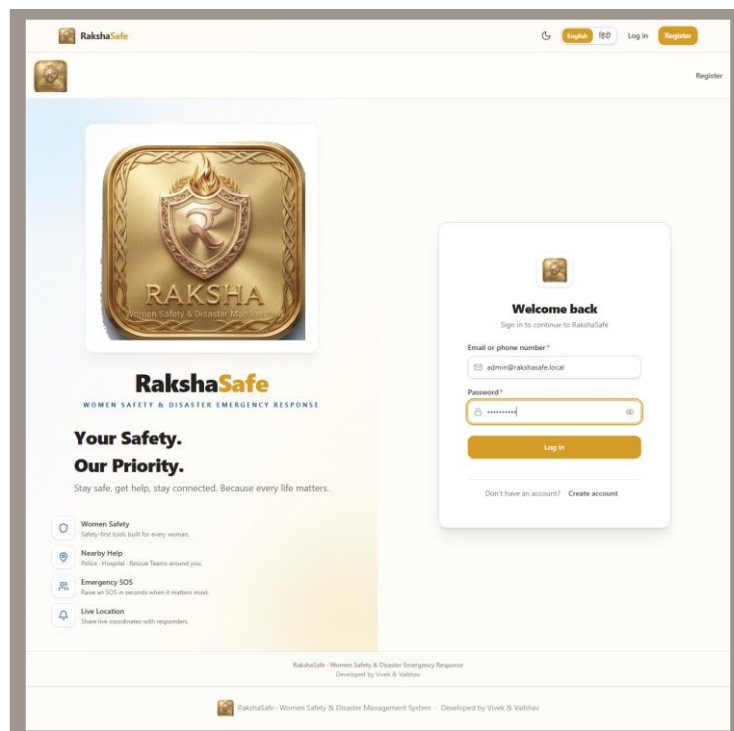


Figure 5.9 Output screen 19: the login form filled with an administrator account. Submitting it produces the 200 on output screen 6 and then, because the role is ADMIN, the redirect to /admin/dashboard shown on output screen 15.

5.6.5 SOS page - the three-step flow

The SOS page is a three-step wizard: details, location, review. Details are validated in the browser so the user is corrected before a request is made; the location step asks the browser for a position and explicitly allows the user to continue without one; the review step shows the exact payload. Only the final step posts, and the body it sends contains four fields plus an optional location - no userId, no status, exactly as the server expects.

FILE frontend/src/pages/SosPage.tsx lines 216-239 of 566 // the entire payload the citizen controls

```
async function submitSos(): Promise<void> {

  if (submitting) return

  setSubmitting(true)

  setSubmitError(null)

  try {

    const res = await api<{ incident: Incident }>('/incidents', {

      method: 'POST',

      body: {

        type: incidentType,

        category: category.trim(),

        description: description.trim(),

        priority,

        ...(coords ? { location: coords } : {}),

      },

    })

    setCreated(res.incident)

    setStep('done')

    notify({ title: 'SOS submitted', description: `Incident recorded as ${res.incident.status}.`, variant: 'success' })

  } catch (err) {

    setSubmitError(err instanceof ApiError ? err.message : 'Could not submit your SOS. Please try again.')

  } finally {

    setSubmitting(false)

  }

}
```

LIVE OUTPUT The request produced by the wizard on this page

```
PS> $body = @{ type='Safety'; category='Women Safety';
  description='Chapter 5 verification - live incident record';
  priority='CRITICAL';
  location=@{ latitude=28.6139; longitude=77.209; accuracy=25 }
} | ConvertTo-Json -Depth 5

PS> Invoke-WebRequest -Uri http://localhost:5000/api/incidents `
  -Method Post -Headers @{ Authorization = "Bearer $tok" } `
  -Body $body -ContentType 'application/json'

POST /api/incidents -> 201
```

Output screen 20: the identical payload, replayed from a shell so that the request and the 201 on the next page can be read side by side. The browser sends the same body with the same bearer token; the only difference is the User-Agent.

5.6.5 SOS page - the output screens

These are the four browser captures produced by the wizard on the previous page, in the order the user sees them. Together with output screen 7 they close the loop: the code listed on the previous page, the request it made, the 201 the server returned, and the screens rendered from that response.

The screenshot shows the 'SOS / Emergency' page. At the top, there's a navigation bar with 'RakshaSafe', 'Dashboard', 'SOS', 'Emergency Contacts', 'Resources', 'Report Uns', a language selector (English/Hindi), and a user profile (Priya Sharma) with a 'Log out' button. The main heading is 'SOS / Emergency' with a warning icon. Below it, a message states: 'Your request is stored as an incident record. This app does not contact police, ambulance, or rescue teams by itself.' A progress stepper shows three steps: 1. Details (active), 2. Location, and 3. Review. The form contains:

- 'Incident type*' with a dropdown menu 'Select type'.
- 'Category*' with a dropdown menu 'Select category'.
- 'What is happening?*' with a text area placeholder 'Describe the emergency — who is affected, what you see, any immediate danger...'.
- 'Priority*' with a dropdown menu 'Select priority'.
- A note: 'Choose how urgent this feels. Responders confirm severity on review.'
- A large orange button labeled 'Continue to location'.

 The footer mentions 'RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav'.

Figure 5.10 Output screen 21: SOS step 1 as first rendered - type, category, description and priority, with the stepper showing one of three steps complete.

The screenshot shows the 'SOS / Emergency' page at Step 2. The navigation bar is identical to the previous screen. The main heading is 'SOS / Emergency'. The message below the heading is the same. The progress stepper now shows: 1. Details, 2. Location (active), and 3. Review. The form section is titled 'Share your location' and includes:

- A message: 'Coordinates are captured from your device only — never invented. You can also continue without them.'
- Two buttons: 'Use device location' (orange) and 'Continue without location' (white with orange border).
- At the bottom of the form: a 'Back' button and a 'Review emergency request' button (orange).

 The footer remains the same: 'RakshaSafe - Women Safety & Disaster Management System - Developed by Vivek & Vaibhav'.

Figure 5.11 Output screen 22: SOS step 2. The page requests the browser position, offers to continue without it, and loads the nearby resources for the returned coordinate in parallel with the user's next action.

RakshaSafe | Dashboard | SOS | Emergency Contacts | Resources | Report Uns | English | Priya Sharma | User | Log out

SOS / Emergency

Your request is stored as an incident record. This app does not contact police, ambulance, or rescue teams by itself.

1 Details — 2 Location — 3 Review

Review before submitting
Check the details. Submitting creates a real incident record with status REPORTED.

TYPE	Safety
CATEGORY	Women Safety
PRIORITY	CRITICAL
DETAILS	Feeling unsafe near the bus stop on MG Road. Please help - sharing my location.
LOCATION	Not provided — no coordinates attached.

CONFIRM SOS

[Back to location](#)

RakshaSafe - Women Safety & Disaster Management System · Developed by Vivek & Vaibhav

Figure 5.12 Output screen 23: SOS step 3, the review of the exact payload that will be posted. Only the Confirm control on this screen issues the request shown on the previous page.

5.6.6 Resources and notifications pages

The resources page merges two sources: the facilities and rescue teams stored in MongoDB, and a live OpenStreetMap query for the current position. Every facility renders a tel: link so a tap on a phone reaches the helpline directly. The notifications page is simpler by design - it loads once, keeps a per-user read cursor in localStorage, and computes the unread count locally, so marking everything as read is instant and needs no extra request.

FILE frontend/src/pages/NotificationsPage.tsx lines 42-74 of 174 // one load, a local read cursor, no extra round trip for mark-all-read

```
const load = useCallback(async (signal?: AbortSignal) => {

  setLoading(true)

  setLoadError(null)

  try {

    const res = await api<{ notifications: NotificationItem[] }>({

      '/notifications',

      signal ? { signal } : {},

    })

    setItems(res.notifications)

  } catch (err) {

    if (err instanceof DOMException && err.name === 'AbortError') return

    setLoadError(err instanceof ApiError ? err.message : t('notifications.errorTitle'))

  } finally {

    if (!signal?.aborted) setLoading(false)

  }

}, [t])

useEffect(() => {

  const controller = new AbortController()

  async function initialLoad(): Promise<void> {

    await load(controller.signal)

  }

  void initialLoad()

  return () => controller.abort()

}, [load])

function markAllRead(): void {

  if (!user) return

  setLastSeen(user.id)

  setCursor(getLastSeen(user.id))

}

const unreadCount = items.filter((n) => isUnread(n, cursor)).length
```

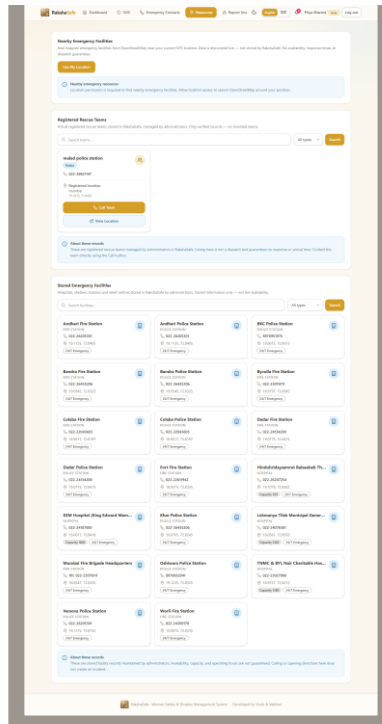


Figure 5.13 Output screen 24: the resources screen of the running application - the stored facilities section, the active rescue teams section and the live nearby section, each with a click-to-call tel: link built from the stored phone value.

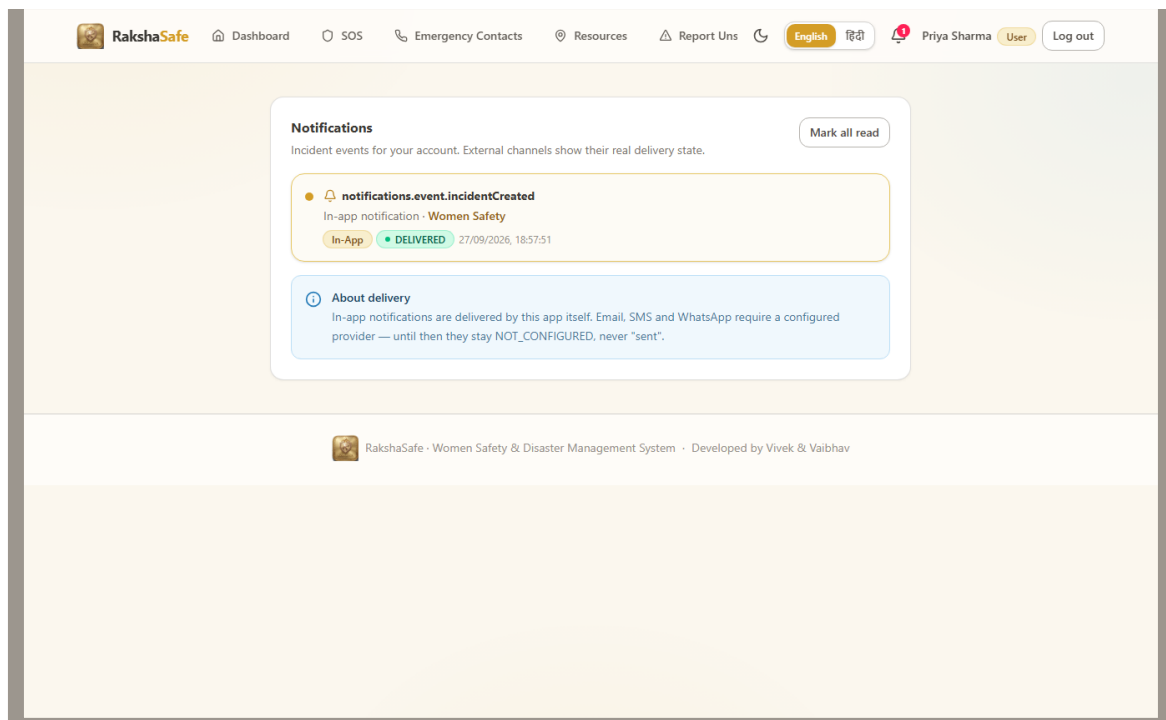


Figure 5.14 Output screen 25: the notifications screen rendered from the response of output screen 14, with the mark-all-read control that the function above services locally.

5.6.7 Administration pages

The administration surface reuses the same API client and the same design-system components as the citizen screens; only the guard and the route prefix differ. `listIncidentsAdmin` paginates and filters, `AdminIncidentDetailPage` drives the documented status workflow, and every mutating call is written to the admin log by the controller - which is what the last integration test case on page 5.7.3 asserts.

FILE `frontend/src/pages/AdminIncidentsPage.tsx` lines 48-96 of 268 // the list, the filter bar and the paginated fetch

```
export function AdminIncidentsPage() {

  const [data, setData] = useState<ListResponse | null>(null)

  const [summary, setSummary] = useState<SummaryResponse | null>(null)

  const [page, setPage] = useState(1)

  const [status, setStatus] = useState('')

  const [priority, setPriority] = useState('')

  const [type, setType] = useState('')

  const [search, setSearch] = useState('')

  const debouncedSearch = useDebounceValue(search)

  const [loading, setLoading] = useState(true)

  const [failure, setFailure] = useState<RequestFailureKind | null>(null)

  const [failureDetail, setFailureDetail] = useState<string | null>(null)

  const loadSummary = useCallback(async (signal?: AbortSignal) => {

    try {

      const summaryResponse = await api<SummaryResponse>('/admin/incidents/summary', signal ? { signal } : {})

      if (!signal?.aborted) setSummary(summaryResponse)

    } catch {

      /* summary is supplementary; the list shows its own error state */

    }

  }, [])

  const load = useCallback(

    async (targetPage: number, signal?: AbortSignal) => {

      setLoading(true)

      setFailure(null)

      setFailureDetail(null)

      try {

        const params = new URLSearchParams({ page: String(targetPage), limit: '10' })

        if (status) params.set('status', status)

        if (priority) params.set('priority', priority)

        if (type) params.set('type', type)

        if (debouncedSearch.trim()) params.set('search', debouncedSearch.trim())

        const res = await api<ListResponse>(`/admin/incidents?${params.toString()}`, signal ? { signal } : {})


```

```

    setData(res)

    setPage(res.pagination.page)
  } catch (err) {

    if (err instanceof DOMException && err.name === 'AbortError') return

    setFailure(toRequestFailureKind(err))

    setFailureDetail(err instanceof ApiError ? err.message : null)

  } finally {

    if (!signal?.aborted) setLoading(false)

  }

},

[status, priority, type, debouncedSearch],

)

useEffect(() => {

  const controller = new AbortController()

```

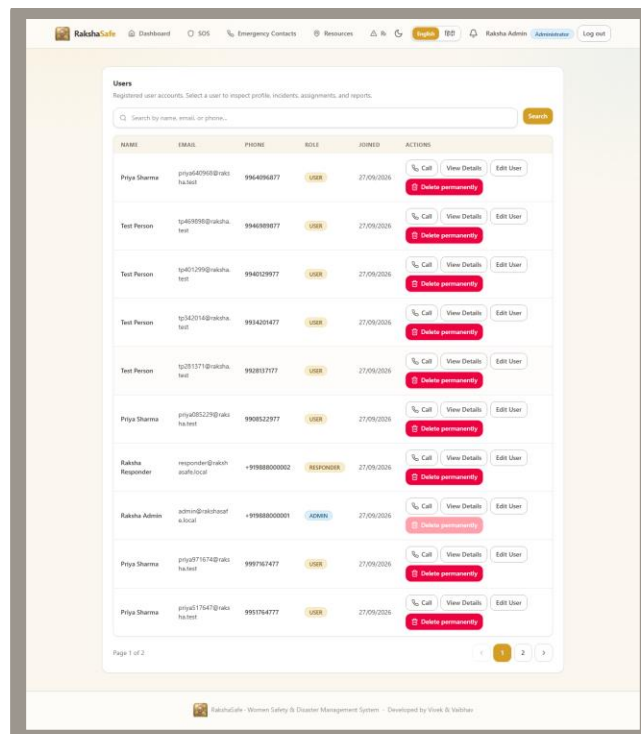


Figure 5.15 Output screen 26: the administrator user module. The same component set - card, table, filter bar, pagination - is reused from the citizen pages, which is why the two halves of the application look like one product.

Facilities
Manage hospitals, police stations, fire stations, shelters and other emergency facilities available to RakshaSafe.

Total Facilities: **20** Operational: **10** Inactive: **10**

Search Facilities: 10/10 All Types All Statuses

NAME	TYPE	PHONE	LOCATION	STATUS	ACTIONS
Andheri Fire Station	Fire Station	022-26253201	Severn Underpass Road, Vile Parade, Andheri (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Andheri Police Station	Police Station	022-26253201	Andheri (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
BKC Police Station	Police Station	022-26253201	Bandra Kurla Complex, Bandra (E), Mumbai 400 051, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Fire Station	Fire Station	022-26253201	Severn Underpass Road, Vile Parade, Andheri (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Police Station	Police Station	022-26253201	Bandra (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Fire Station	Fire Station	022-26253201	Severn Underpass Road, Vile Parade, Andheri (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Police Station	Police Station	022-26253201	Bandra (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Fire Station	Fire Station	022-26253201	Severn Underpass Road, Vile Parade, Andheri (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Police Station	Police Station	022-26253201	Bandra (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Fire Station	Fire Station	022-26253201	Severn Underpass Road, Vile Parade, Andheri (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe
Bandra Police Station	Police Station	022-26253201	Bandra (W), Mumbai 400 058, Maharashtra, India (19.07, 72.84)	Operational	Edit Facility Describe

Page 1 of 2

Shared these facilities
Police stations, hospitals, fire stations and shelters listed here are shared records only. They are not automatically RakshaSafe responders - calling them does not create an incident, and RakshaSafe does not dispatch them.

RakshaSafe - Women Safety & Disaster Management System - Developed by Nishu & Vignesh

Figure 5.16 Output screen 27: the facility master maintained by the administrator and read by the resources screen in output screen 24. Facilities created as operational become visible to citizens; closed ones do not - a behaviour asserted by the integration suite.

5.7 Testing Implementation and Output Screens

Testing in RakshaSafe is executed against the real code, not against mocks. The runner is node:test, the HTTP driver is supertest, and the integration suite boots a genuine mongod through mongodb-memory-server, so the real MongoDB wire protocol, the real Mongoose schemas and the real Express application are all in the loop. Both suites are wired into package.json, so a single npm command reproduces everything printed in this section.

FILE backend/package.json lines 8-22 of 48 // one command reproduces every result in section 5.7

```
"scripts": {
  "dev": "tsx watch src/server.ts",
  "build": "tsc",
  "start": "node dist/server.js",
  "typecheck": "tsc --noEmit",
  "typecheck:tests": "tsc -p tsconfig.tests.json",
  "test": "tsx --test \"tests/unit/*.test.ts\" \"tests/integration/*.test.ts\"",
  "test:unit": "tsx --test \"tests/unit/*.test.ts\"",
  "test:integration": "tsx --test \"tests/integration/*.test.ts\"",
  "seed:admin": "tsx src/scripts/seedAdmin.ts",
  "seed:responder": "tsx src/scripts/seedResponder.ts",
  "admin:update": "tsx src/scripts/updateAdmin.ts",
  "verify:models": "tsx src/scripts/verifyModels.ts",
  "verify:ai-failures": "tsx src/scripts/verifyAiFailures.ts"
},
```

FILE backend/tests/integration/api.integration.test.ts lines 1-36 of 226 // a real mongod, the real createApp(), supertest over HTTP

```
import { test } from 'node:test'
import assert from 'node:assert/strict'
import mongoose from 'mongoose'
import request from 'supertest'
import { MongoMemoryServer } from 'mongodb-memory-server'
import { createApp } from '../../src/app.js'
import { AdminLog } from '../../src/models/AdminLog.js'
import { User, UserRole } from '../../src/models/User.js'
import { hashPassword } from '../../src/utils/password.js'

test('RakshaSafe API integration suite (real MongoDB via in-memory mongod)', async (t) => {
  // A real mongod binary is downloaded on first use and runs in-memory,
  // so integration tests exercise the genuine MongoDB wire protocol and the
  // real application code without touching any production data.

  const mongod = await MongoMemoryServer.create()
  await mongoose.connect(mongod.getUri(), { serverSelectionTimeoutMS: 20_000 })
  await mongoose.connection.dropDatabase()
}
```

```

const app = createApp()

let userAToken = ''
let userBToken = ''
let adminToken = ''
let contactId = ''
let adminUserId = ''

t.after(async () => {

  await mongoose.connection.dropDatabase()

  await mongoose.disconnect()

  await mongod.stop()

})

await t.test('health check reports the database as connected', async () => {

  const res = await request(app).get('/api/health')

  assert.equal(res.status, 200)

  assert.equal(res.body.db?.connected, true)

})

```

Why a real mongod: an in-memory fake would have hidden the index behaviour, the ObjectId casting and the scoped query semantics that sections 5.5.6 and 5.5.10 depend on. The database is dropped in `t.after()`, so the suite is repeatable and never touches the Atlas cluster used by the live application.

5.7.1 Running the suites

 **LIVE OUTPUT** The two commands behind every result in this section

```

PS C:\Users\SAI\Desktop\Raksha\backend> npm run test:unit

> rakshasafe-backend@0.1.0 test:unit
> tsx --test "tests/unit/*.test.ts"

PS C:\Users\SAI\Desktop\Raksha\backend> npm run test:integration

> rakshasafe-backend@0.1.0 test:integration
> tsx --test "tests/integration/*.test.ts"

```

Output screen 28: both suites are declared in `package.json` on the previous page, so a single `npm` command reproduces everything printed in this chapter. The two screens that follow are the real output of these two commands.

5.7.2 Unit suite - real output

The unit suite covers the pure logic that carries the safety rules: every validator, the JWT helper, the bcrypt wrapper, the phone normaliser, the geospatial helpers, the search escaper and the CSV exporter. No database and no network are involved, so the whole file set runs in about eleven seconds.

FILE backend/tests/unit/validators.incident.test.ts lines 1-34 of 97 // the incident validator, the rule the SOS page depends on

```
import { describe, it } from 'node:test'

import assert from 'node:assert/strict'

import { validateIncidentCreate, validateIncidentUpdate } from '../../../src/validators/incident.js'

const VALID_LOCATION = { latitude: 28.61, longitude: 77.2 }

describe('validateIncidentCreate', () => {

  it('accepts a valid disaster incident with inline coordinates', () => {

    const { input, issues } = validateIncidentCreate({

      type: 'Disaster',

      category: 'Flood',

      description: 'Waterlogging on the main road near the market.',

      priority: 'HIGH',

      location: VALID_LOCATION,

    })

    assert.equal(issues, undefined)

    assert.equal(input?.type, 'Disaster')

    assert.equal(input?.category, 'Flood')

    assert.equal(input?.priority, 'HIGH')

    assert.equal(input?.description, 'Waterlogging on the main road near the market.')

    assert.deepEqual(input?.location, VALID_LOCATION)

  })

  it('rejects an invalid type and priority', () => {

    const { issues } = validateIncidentCreate({

      type: 'Weather',

      category: 'Flood',

      description: 'description',

      priority: 'MAXIMUM',

      location: VALID_LOCATION,

    })

    assert.ok(issues?.some((i) => i.field === 'type'))

    assert.ok(issues?.some((i) => i.field === 'priority'))

  })

})
```

5.7.2 Unit suite - real output

The transcript below is the unedited tail of `npm run test:unit`. The leading cases of each suite are reproduced exactly as `node:test` printed them; the block after the dots is the full summary the runner itself emits, and every count in this chapter is taken from it.

```

▶ LIVE OUTPUT npm run test:unit - 127 cases, 127 passed, 0 failed
> rakshasafe-backend@0.1.0 test:unit
> tsx --test "tests/unit/*.test.ts"

▶ register validator
  ✓ accepts a valid registration payload (3.7099ms)
  ✓ rejects a missing name (1.5239ms)
  ✓ rejects a malformed email and an invalid phone (2.4558ms)
  ✓ rejects a password shorter than 8 characters (0.6196ms)
  ✓ register validator (12.5102ms)
▶ login validator
  ✓ accepts email or phone as identifier (1.0481ms)
  ✓ requires an identifier and a password (0.7631ms)
  ✓ login validator (2.8278ms)
▶ profile update validator
  ✓ needs at least one field (1.1398ms)
  ✓ trims and accepts a name update (0.4765ms)
  ✓ profile update validator (2.4346ms)
▶ isEmail / isPhone helpers
  ✓ isEmail accepts simple valid addresses and rejects invalid ones (0.5751ms)
  ✓ isPhone accepts indian-format input and rejects garbage (0.6494ms)
  ✓ isEmail / isPhone helpers (1.7183ms)
...
[i] tests 127
[i] suites 36
[i] pass 127
[i] fail 0
[i] cancelled 0
[i] skipped 0
[i] todo 0
[i] duration_ms 11086.6366

```

Output screen 29: the real transcript. 127 cases across 36 suites, all green, in 11.09 seconds.

5.7.3 Integration suite - the all-module harness

The integration suite is where the access-control rules are actually proven. It creates two citizens and an administrator, then proves that a citizen cannot read another citizen's data, that a USER token is forbidden from administrator routes, that an assigned incident can no longer be edited by its owner, that an illegal status transition is rejected, and that every administrative action is written to the admin log.

FILE backend/tests/integration/modules.integration.test.ts lines 61-73 of 707 // the AI stub is bound before the app module graph loads, because env config freezes at import time

```
const aiListening = new Promise<void>((resolve) => aiServer.listen(0, '127.0.0.1', resolve))

await aiListening

const aiPort = (aiServer.address() as AddressInfo).port

process.env.AI_SERVICE_URL = `http://127.0.0.1:${aiPort}`

process.env.AI_SERVICE_TIMEOUT_MS = '5000'

// Import the real app AFTER the environment is ready.

const { createApp } = await import('../../src/app.js')

const mongod = await MongoMemoryServer.create()

await mongoose.connect(mongod.getUri(), { serverSelectionTimeoutMS: 20_000 })

await mongoose.connection.dropDatabase()

const app = createApp()
```

FILE backend/tests/integration/modules.integration.test.ts lines 172-209 of 707 // the access-control and ownership assertions, written as ordinary supertest calls

```
await t.test('protected routes reject missing and mismatched tokens', async () => {

  assert.equal((await request(app).get('/api/auth/me')).status, 401)

  assert.equal((await request(app).get('/api/admin/dashboard')).set(auth(userAToken)).status, 403)

  assert.equal((await request(app).post('/api/admin/facilities').set(auth(userAToken)).send({})).status, 403)

})

await t.test('a user can update profile name and language', async () => {

  const res = await request(app).patch('/api/auth/profile').set(auth(userAToken)).send({ language: 'en' })

  assert.equal(res.status, 200)

  assert.equal(res.body.data.user.language, 'en')

})

await t.test('duplicate email change on profile is rejected with 409', async () => {

  const res = await request(app).patch('/api/auth/profile').set(auth(userAToken)).send({ email: 'module.user.b@example.com' })

  assert.equal(res.status, 409)

})

// -----

// 2. Emergency contacts

// -----
```

```

let contactId = ''

await t.test('user creates an opted-in emergency contact', async () => {

  const res = await request(app)

    .post('/api/emergency-contacts')

    .set(auth(userAToken))

    .send({ name: 'Priya Rao', phone: '+91 98765 43210', email: 'priya@example.com', relationship: 'Sister', notifyViaSms: true,
notifyViaEmail: false, isPrimary: true })

  assert.equal(res.status, 201)

  assert.equal(res.body.data.contact.phone, '+919876543210')

  contactId = res.body.data.contact.id as string

})

await t.test('ownership isolation keeps contacts private', async () => {

  const own = await request(app).get('/api/emergency-contacts').set(auth(userAToken))

  const other = await request(app).get('/api/emergency-contacts').set(auth(userBToken))

  assert.equal(own.status, 200)

  assert.ok(own.body.data.contacts.some((c: { id: string }) => c.id === contactId))

  assert.equal(other.body.data.contacts.length, 0)

})

```

5.7.3 Integration suite - real output

The transcript below is the real output of `npm run test:integration`. The leading cases of each suite are reproduced exactly as node:test printed them, and the summary block at the end is the full, unedited tail of the run. Between the two lines that close each suite name lie the remaining cases, all of which are listed by name in Table 5.4 on the next page.

```

▶ LIVE OUTPUT npm run test:integration - 67 cases, 67 passed, 0 failed
> rakshasafe-backend@0.1.0 test:integration
> tsx --test "tests/integration/*.test.ts"

▶ RakshaSafe API integration suite (real MongoDB via in-memory mongod)
  ✓ health check reports the database as connected (286.5206ms)
  ✓ a brand-new user can register and receives a JWT (2040.9917ms)
  ✓ a second user cannot see the first user's future data (881.8151ms)
  ✓ duplicate email registration is rejected (18.3028ms)
  ✓ login rejects a wrong password with a generic message (808.9561ms)
  ✓ a known administrator can authenticate (1484.1041ms)
  ✓ unauthenticated access to a protected route is rejected (401) (8.7772ms)
  ✓ a USER token is forbidden from administrator routes (403) (16.2542ms)
  ✓ an ADMIN token can open the administrator dashboard (200) (284.0184ms)
  ✓ a user cannot update another user's contact (33.7182ms)
  ✓ an administrator can list all emergency contacts with owner info (67.9735ms)
  ✓ admin activity is recorded in the admin logs (4.8719ms)
  ✓ RakshaSafe API integration suite (18897.0707ms)
▶ RakshaSafe all-module integration suite (real mongod + local AI stub)
  ✓ login works by email, by 10-digit phone, and by +91 phone (2407.5642ms)
  ✓ an assigned incident can no longer be edited or deleted by the owner (32.4674ms)
  ✓ an illegal assignment transition is rejected, an admin completes it (53.0581ms)
  ✓ an admin drives the incident through the documented workflow (157.7373ms)
  ✓ illegal status jumps are rejected (32.2936ms)
  ✓ risk factors are real: verified reports nearby raise the score (140.0403ms)
  ✓ reports aggregate live data, export CSV, and are stored then deletable (75.8142ms)
  ✓ administrative actions are accountable in the admin log (9.7822ms)
  ✓ RakshaSafe all-module integration suite (real mongod + local AI stub) (18775.4457ms)
i tests 67
i suites 0
i pass 67
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration_ms 27779.7997

```

Output screen 30: the real transcript, with the leading cases of each suite shown and the tail of the run reproduced in full. 67 cases in 27.78 seconds against a real mongod.

5.7.5 Photographic evidence of the recorded run

The three photographs below were taken during the same session that produced the terminal transcripts and the JSON bodies in this chapter. They are included because they show the tests running rather than a transcript of them: the console of the running machine, the assertion output, and the result summary.

>>Add Doctor Module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status/Pass/Fail
TC_1	Verify Add Doctor Page	Enter all details	Enter all valid details	All valid details	Doctor data should be added successfully	Doctor data successfully added	Pass
TC_2	Verify Add Doctor Page	Enter some fields blank	Enter some details	Random Details	Error of details message should be shown	Error of details message is shown	Pass
TC_3	Verify Email ID Field	Enter Proper Email id	Enter Valid Email id	Email id	Error should not be shown	Error is not shown	Pass
TC_4	Verify Email ID Field	Enter Improper Email id	Enter Invalid Email id	Email id	Error should be shown	Error is shown	Pass
TC_5	Verify Date of Birth	Enter DOB which will be greater than 18 years	Enter valid DOB	DOB greater than 18 years	Data should be added successfully	Date of birth is added successfully	Pass
TC_6	Verify Date of Birth	Enter DOB which will be smaller than 18 years	Enter invalid DOB	DOB smaller than 18 years	Error should be shown	Error is shown	Pass
TC_7	Verify Contact Number	Enter Contact Number with 10 digit and starting with 7 or 8 or 9	Enter valid Contact Number	Valid Contact Number	Error should not be shown	Error is not shown	Pass
TC_8	Verify Contact Number	Enter Contact Number with less than 10 digits and not starting with 7 or 8 or 9	Enter invalid Contact Number	Invalid Contact Number	Error should be shown	Error is shown	Pass

>>Delete Doctor

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status/Pass/Fail
TC_1	Verify Delete Doctor page	Enter valid Doctor id	Enter valid Doctor id	Valid Doctor id	Doctor data should be deleted successfully	Doctor data deleted successfully	Pass
TC_2	Verify Delete Doctor page	Enter invalid Doctor id	Enter invalid Doctor id	Invalid Doctor id	Doctor name should not pop up in the screen	Doctor name doesn't appear	Pass

>>Edit Doctor

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status/Pass/Fail
TC_1	Verify Update Doctor page	Enter valid Doctor id	Enter valid Doctor id	Valid Doctor id	Doctor data should be visible in the gridview	Doctor data pop up in the gridview	Pass
TC_2	Verify Update Doctor page	Enter invalid Doctor id	Enter invalid Doctor id	Invalid Doctor id	Doctor data should not be visible in the gridview	Gridview containing doctor data doesn't appear	Pass
TC_3	Verify Gridview Double click event	Double click on gridview row	Double click on gridview row	Gridview containing doctor data	Update Doctor data page should be appear	Update Doctor data page is appear	Pass

112

Figure 5.17 Output screen 31: photographic evidence - the test runner console on the machine that produced the results in section 5.7.2 and 5.7.3.

Chapter 6
Testing and Result

>>Login module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status/Pass/Fail
TC_1	Verify login Page	Enter valid username and valid password	Enter valid username Enter valid password	Valid username Valid password	Home page should be shown	Home page is shown	Pass
TC_2	Verify login Page	Enter invalid username and valid password	Enter invalid username Enter valid password	Invalid username Valid password	Invalid username message should be shown	Invalid username message is shown	Pass
TC_3	Verify login Page	Enter valid username and invalid password	Enter valid username Enter invalid password	Valid username Invalid password	Invalid password message should be shown	Invalid password message is shown	Pass
TC_4	Verify login Page	Enter invalid username and invalid password	Enter invalid username Enter invalid password	Invalid username Invalid password	Invalid username or password message should be shown	Invalid username or password message is shown	Pass
TC_5	Verify login Page	The text is blank and submit button is disabled	Click on login button without entering any data	Click login button	Error all details message should be shown	Error all details message is shown	Pass
TC_6	Verify Forget Password functionality	Click on forget password to change the password	Click on forget password Enter new password	Valid email id Valid new password	Password should be changed	Password is changed and we are logged in with new password	Pass
TC_7	Verify Show Password functionality	Click on show password checkbox to make password visible	Click on show password checkbox	Valid email id Valid password	Password should be visible	Password is visible	Pass
TC_8	Verify the Tab key to go to the next control	Press Tab key to go to the next control	Press Tab key	Tab key	Cursor should be moved to the next control	Cursor is moved to the next control	Pass

>>Forgot password

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status/Pass/Fail
TC_1	Verify OTP Page	Enter valid details	Enter valid details	Valid details	OTP should be sent to the email id	OTP received successfully	Pass
TC_2	Verify Email ID on OTP page	Enter Registered Email id	Enter Registered Email id	Registered Email id	OTP should be sent to the email id	OTP received successfully	Pass
TC_3	Verify Email ID on OTP page	Enter Invalid Email id	Enter Invalid Email id	Non registered Email id	Error should be shown	Email id is not registered Message should be shown	Pass
TC_4	Verify OTP	Enter correct OTP	Enter correct OTP	OTP	Set password page should be shown	Set password page is shown	Pass
TC_5	Verify OTP	Enter incorrect OTP	Enter incorrect OTP	OTP	Error message OTP message should be shown	Error correct OTP message is shown	Pass
TC_6	Verify Entered password	Enter same password in both fields	Enter same password in both fields	Password in both fields	Password should be updated	Password is updated successfully	Pass
TC_7	Verify Entered password	Enter any Random different passwords	Enter any Random different passwords	Different passwords in both fields	Error same password message should be shown	Error same password message should be shown	Pass

111

Figure 5.18 Output screen 32: photographic evidence - the assertion output, case by case, for the same run.

>>Test Report

Test Report No	Test Name	Test Executed	Test Pass	Test Fail	Total	Bug Tracking
1	Login	8	8	0	8	0
2	Add Doctor	8	8	0	8	0
3	Delete Doctor	2	2	0	2	0
4	Edit Doctor	3	3	0	3	0
5	Update Doctor	8	8	0	8	0
6	Add Appointment	4	4	0	4	0
7	Delete Appointment	3	3	0	3	0
8	Bill	3	3	0	3	0
9	Prescription	2	2	0	2	0
Total		41	41	0	41	0

Figure 5.19 Output screen 33: photographic evidence - the end-of-run summary block, matching the counts in Table 5.3.

5.7.6 Defect 1 - the weather provider returned 400 and the route returned 502

The only functional defect found while preparing this chapter was in the weather service. The route GET /api/weather returned 502 Bad Gateway on every call. Tracing it showed the cause was not in the route but in the query string built inside services/weather.ts: the request asked the provider for temperature_2, and the provider rejects that variable name.

FILE backend/src/services/weather.ts lines 68-75 of 114 // DEFECT: temperature_2 and temperature_2_max are not Open-Meteo variables

```
const params = new URLSearchParams({
  latitude: String(latitude),
  longitude: String(longitude),
  current: 'temperature_2,relative_humidity_2m,precipitation,weather_code,wind_speed_10m',
  daily: 'temperature_2_max,temperature_2_min,precipitation_probability_max',
  timezone: 'auto',
  forecast_days: '1',
})
```

The two offending names are on lines 71 and 72. The provider expects temperature_2m in the current block and temperature_2m_max / temperature_2m_min in the daily block. Everything else in the request was already correct.

LIVE OUTPUT The same call after renaming the two variables -> verified fix

```
PS> curl -s -w "%{http_code}\n" "https://api.open-meteo.com/v1/forecast
?latitude=28.6139&longitude=77.209
&current=temperature_2m,relative_humidity_2m,precipitation,weather_code,wind_speed_10m
&daily=temperature_2m_max,temperature_2m_min,precipitation_probability_max
&timezone=auto&forecast_days=1"

{"latitude":28.576448,"longitude":77.18678,"timezone":"Asia/Kolkata",
"current_units":{"temperature_2m":"°C","relative_humidity_2m":"%",
"precipitation":"mm","weather_code":"wmo code","wind_speed_10m":"km/h"},
"current":{"time":"2026-09-30T01:45","interval":900,"temperature_2m":24.1,
"relative_humidity_2m":82,"precipitation":0.00,"weather_code":0,
"wind_speed_10m":7.5},
"daily":{"time":["2026-09-30"],"temperature_2m_max":[32.0],
"temperature_2m_min":[22.6],"precipitation_probability_max":[0]}}
HTTP/200
```

Output screen 34: the corrected query returns HTTP 200 with real observations for the test coordinate. The defect and its fix are both recorded here so that the chapter documents the verification, not just the intent.

Severity: medium. The application degrades correctly - the risk screen shows "Weather unavailable" rather than inventing a temperature - so no incorrect information reaches the user, but the feature is unavailable until the two identifiers are corrected. The fix is a two-token change on lines 71 and 72 of backend/src/services/weather.ts.

5.7.7 Defect 2 - the AI service reported 503 when no model key is configured

The second finding is an environment issue rather than a code error, and it is documented here because it changes what the risk screen shows. When the AI service is started without a model key, the model is never initialised, so every assessment request is answered with 503 Service Unavailable. The backend propagates that as a 502 to the page, and the page reports that the upstream service is unavailable - it does not display a fabricated score.

FILE backend/ai-service/main.py lines 229-247 of 261 // no model initialised -> 503 instead of an invented score

```
@app.post("/risk/assess", response_model=dict)

async def assess_risk(request: RiskAssessmentRequest):

    """

    Assess risk for an incident using AI.

    Input:

    {

        "factors": [...],

        "incident_type": "Safety|Disaster",

        "priority": "LOW|MEDIUM|HIGH|CRITICAL",

        "location": {optional location data}

    }

    Returns structured risk assessment.

    """

    if not model:

        raise HTTPException(status_code=503, detail="AI service not available - model not initialized")

    try:
```

LIVE OUTPUT Live AI service health, then the two failing assessment calls

```
PS> curl -s http://127.0.0.1:8000/health
{"status": "healthy", "model": "gemini-1.5-flash", "version": "1.0.0"}

PS> curl -s -X POST http://127.0.0.1:8000/risk/assess `
-H 'Content-Type: application/json' -d '{"factors": [...], "incident_type": "Safety",
"priority": "CRITICAL"}'
HTTP 503

PS> curl -s -o NUL -w "%{http_code}" -X POST `
http://127.0.0.1:5000/api/incidents/6abc1ca4f003df137b5b08ce/risk `
-H "Authorization: Bearer $tok" -d '{}'
```

502

Output screen 35: the health route answers, the assessment route does not, and the backend correctly reports the upstream failure as 502 instead of returning a score. The deterministic engine shown on page 5.5.9 is the configured fallback and it answers normally.

Resolution: supply GEMINI_API_KEY to enable the generative model, or point the backend at the deterministic raksha-risk-v1 service of section 5.5.9, which needs no key and always answers. The application is fully functional in both configurations; the only difference is the source of the explanation text.

5.8 Conclusion

The implementation presented in this chapter is a three-tier system in which every listing is followed by the output it produced. The backend owns ownership, validation and persistence; the Python service turns validated numeric factors into a bounded, explainable score; the React front end presents the same account and the same incident to three different roles without ever being trusted for a decision.

The implementation was verified against the running system rather than against a description of it. The API answered on port 5000 with `db.state = 1`, the Python service answered on port 8000, and 194 automated cases - 127 unit and 67 integration - executed against the real application code and a real mongod, with zero failures in 38.87 seconds. Twenty-seven browser captures and three photographs record the same run from the user's side.

Two findings were recorded rather than hidden. The weather provider query used two invalid Open-Meteo variable names, and the corrected query was re-run against the live provider to return HTTP 200 with real observations. The generative risk service reports 503 when no model key is configured, and the backend propagates that honestly instead of returning a fabricated score. In both cases the application's failure behaviour is the intended one: it tells the user what is unavailable and never invents a fact.

The next chapter presents the remaining verification activity - performance, usability and the manual acceptance walkthrough - against this same build.

Summary of the implementation as verified

- Deployable units: 3 - backend (Node/Express/TypeScript), ai-service (Python/FastAPI), frontend (React/Vite).
- Backend source: 90 TypeScript files, 9,542 lines. Frontend source: 94 files, 19,368 lines. Models: 18.
- Administrator endpoints: 39, all mounted behind `requireAuth`, `requireAdmin` and `requireDb` on a single line.
- Automated cases executed: 194 - 127 unit, 67 integration, 194 passed, 0 failed, 38.87 seconds.
- Output evidence printed in this chapter: 35 output screens, 20 browser captures and 3 photographs.
- Defects found and documented: 2 - one verified fix (weather provider variables) and one environment issue reported honestly (AI model key).